VialSense Runbook

This runbook installs, tests, and runs the integrated color-analysis prototype. Commands start from the project root. Python 3.10 or newer is recommended.

1. Create the Python environment

macOS / Linux:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -r requirements-dashboard.txt
```

Windows PowerShell:

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -r requirements-dashboard.txt
```

2. Run the tests

macOS / Linux:

```
PYTHONPATH=src python -m pytest -q
```

PowerShell:

```
$env:PYTHONPATH = "src"
python -m pytest -q
```

3. Rebuild the saved Dataset 2 results

Send output to a review directory so the checked-in results are not overwritten:

```
PYTHONPATH=src python -m vialsense.corrected_cli build --output-dir /tmp/vialsense-review
```

This rebuilds Phase 1 timelines and Phase 2 provisional profiles from the saved Dataset 2 RGB/HSV readings and corrected labels.

4. Replay automatic live tracking on Dataset 2 Video 5

Video 5 is not included in the repository. With a local copy of the original MP4:

```
PYTHONPATH=src python -m vialsense.live_cli run \
  --video /path/to/vial_fill_video_5.mp4 \
  --recipes "5x3,6x3,7x3,8x3,9x3,10x3,11x3,12x3,13x3,14x3" \
  --fps 5 --no-window --output-dir /tmp/vialsense-live-review
```

This detects vials, assigns recipes from the operator queue, estimates fill progress from the detected liquid rise, calculates RGB/HSV, and scores usable readings with Phase 2. The output folder includes frame readings, per-vial summaries, events, and a run summary. Use the correct queue for the video; queue misalignment shifts recipe assignments.

PowerShell uses `$env:PYTHONPATH = "src"` on its own line before the command.

5. Launch the Yen dashboard

```
python -m streamlit run app.py
```

Allow camera access in the browser. Select **Automatic vial tracking**, enter the recipe queue in production order, and include the preceding recipe if it is known. Use **Skip missed vial** or **Undo queue step** if vial counting loses alignment. **Manual fallback** lets an operator provide recipe and fill progress when automatic tracking is not reliable.

The liquid finder expects the Dataset 2 camera geometry. For a different camera view, create a calibration JSON with `vialsense.live_cli calibrate`, inspect the generated preview, and upload the JSON in the dashboard. Calibrate from a machine where the target camera is accessible. If the holder is not aligned in the corrected preview, do not rely on the readings.

6. Run the saved integration workflow

Saved detector readings and Video 5 labels can be replayed without the original video:

```
PYTHONPATH=src python -m vialsense.integration_cli saved \
  --output-dir /tmp/vialsense-saved-review
```

For full setup details and limitations, see DASHBOARD_SETUP.md, LIVE_MODE.md, and the project README.

What the results mean

Recipe profiles and anomaly scores are exploratory. Dataset 2 does not provide confirmed contamination labels, so `FLAG` or `review_unusual` requests an operator review and does not confirm contamination or product failure. RGB/HSV values depend on lighting, camera alignment, reflections, fill level, and the finder's selected region. The Video 5 replay supports the Dataset 2 view only; the CSI camera and browser-camera workflow still need testing on the actual setup.